00 01 02 03 04 **05** 06 07 08 09 0A 0B 0C 0D

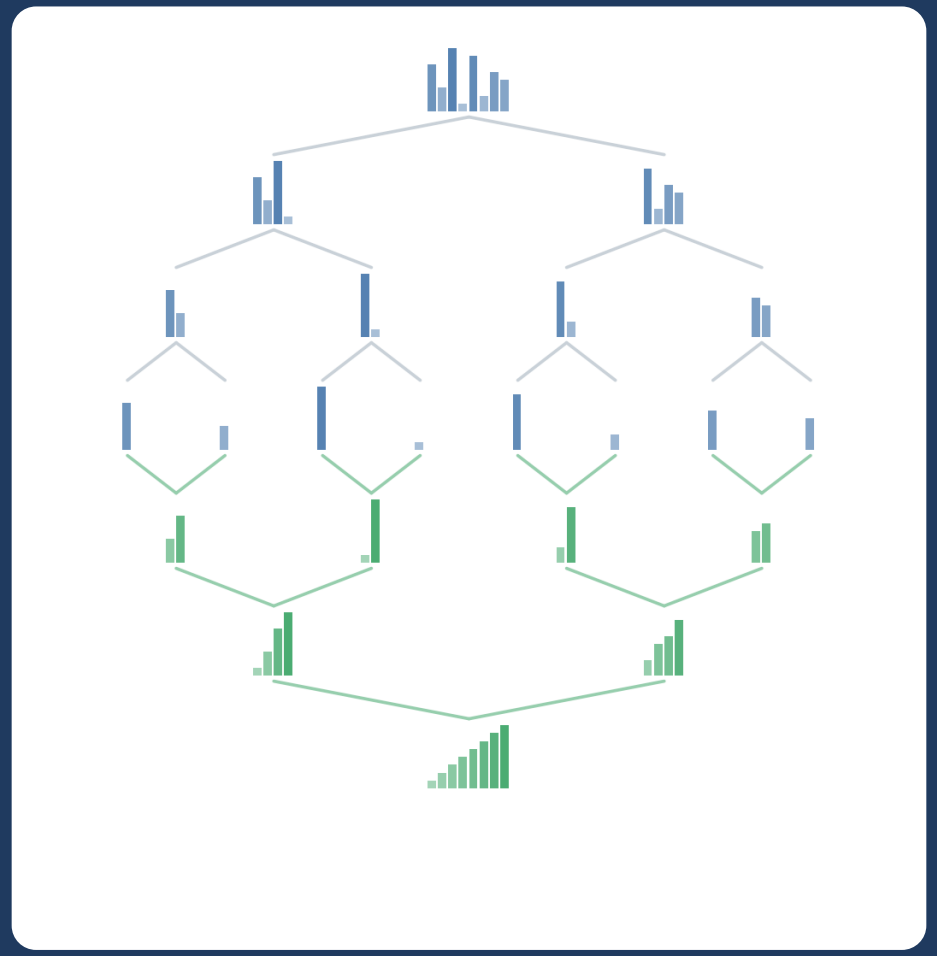
ALGORİTMA VE PROGRAMLAMA – II

HAFTA 06

Böl ve Fethet

Birleştirmeli Sıralama, Hızlı Sıralama ve Yineleme Bağıntıları

MODÜL II · ARAMA VE SIRALAMA



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** **Böl ve Fethet**	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

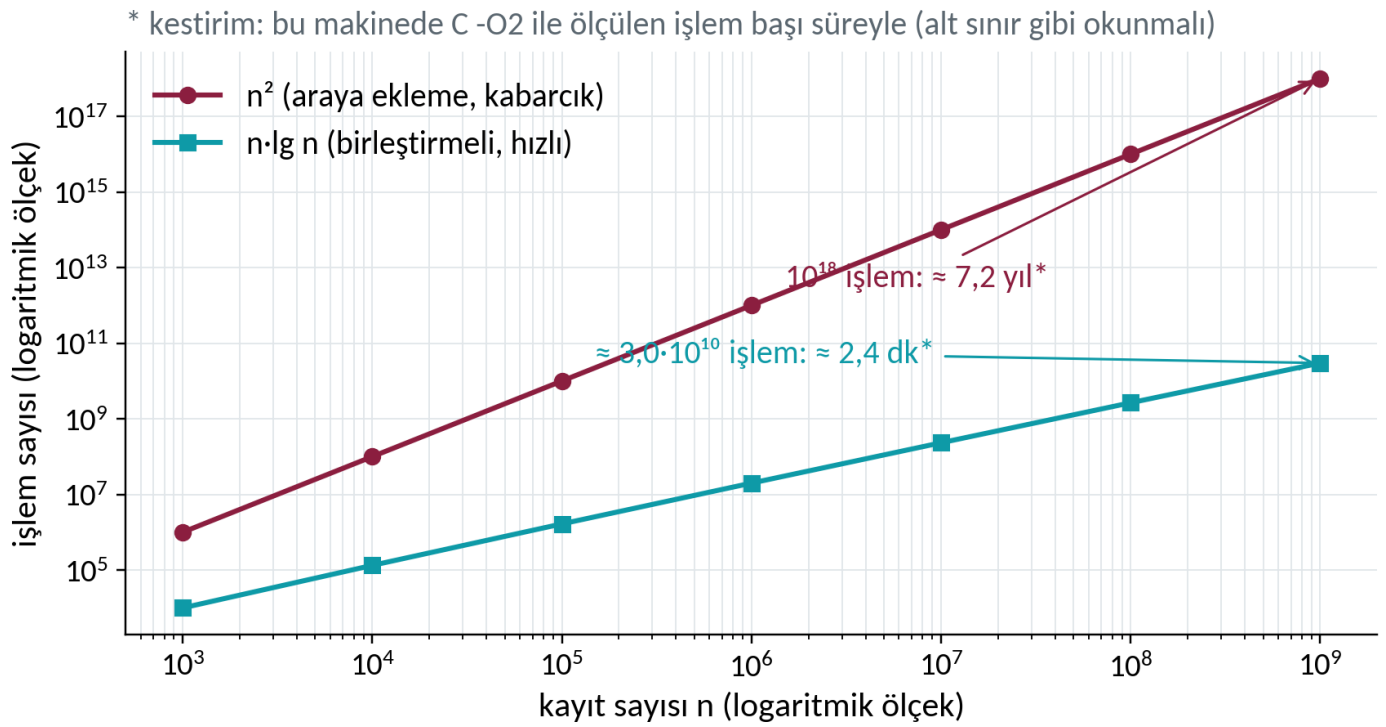
Geçen haftadan üç soru

- **S1 (H5).** Araya ekleme sıralaması [6, 3, 8, 1, 7, 2, 5, 4] dizisini sıralarken kaç kaydırma yapar? Bu sayı neye eşittir?
- **S2 (H4).** 1024 elemanlı sıralı dizide ikili arama en kötü durumda kaç kez ortadaki elemana bakar? Bağıntısı nedir?
- **S3 (H3).** $T(n) = 2T(n - 1) + 1$, $T(0) = 0$ (Hanoi) bağıntısının çözümü nedir? $T(10)$ kaçtır?

Yanıtlar: üç bağıntı, üç büyüme hızı

- **S1:** 16 kaydırma = dizideki ters çevrim sayısı. Bu hafta aynı sayıyı birleştirmeli sıralamayla Θ(n log n)'de sayacağız (Örnek 8).
- **S2:** T(n) = T(n/2) + 1 → lg n + 1; n = 1024 için **11** (Kavram 2 aynı bağıntıyı özyinelemeyle hesaplar).
- **S3:** yerine koyma: T(n) = 2^n − 1; T(10) = **1023**. “Bir eksilten” iki çağrı üstel büyür.
- Bu hafta: **yarıya bölen** iki çağrı + doğrusal iş → **n log n**.

Büyük Soru: 10^{18} mi, $3 \cdot 10^{10}$ mu?



- Bir milyar kaydı sıralamak neden bir milyar kere bir milyar işlem gerektirmez ve “yarıya bölmek” bu farkı nasıl yaratır?
- $n = 10^9$: $n^2 = 10^{18}$ işlem; $n \cdot \lg n \approx 2{,}99 \cdot 10^{10}$ işlem.
- Bu makinenin hızıyla (C, -O2): hızlı sıralama ≈ **2,4 dk**, araya ekleme ≈ **7,2 yıl** (kestirim, alt sınır).
- Ölçülen $n = 10^7$: birleştirmeli 1,51 s, hızlı 1,11 s; araya ekleme ≈ 6,3 sa (kestirim).

GERÇEK DÜNYADAN

1945–1999

Moskova 1959: sözlükteki sözcükleri sıralamak

- Hoare, Moskova Devlet Üniversitesi'nde makine çevirisi üzerinde çalışırken bir cümlenin sözcüklerini sıralayıp manyetik banttaki sözlükte **tek geçişte** aramak istedi; ilk fikri (kabarcık) yavaştı, ikincisi **hızlı sıralama** oldu.
- Elliott Brothers'ta altı penilik iddiayı kazandı; yöntemi ancak 1961'de ALGOL 60'ın **özyinelemesini** öğrenince kolayca yazabildi. Aynı sayıda FIND (quickselect) da yayımlandı.
- Birleştirmeli sıralama 1945'te **von Neumann**'a atfedilir. McIlroy (1999) “katil rakip”le qsort'ları karesel davranışa zorladı.

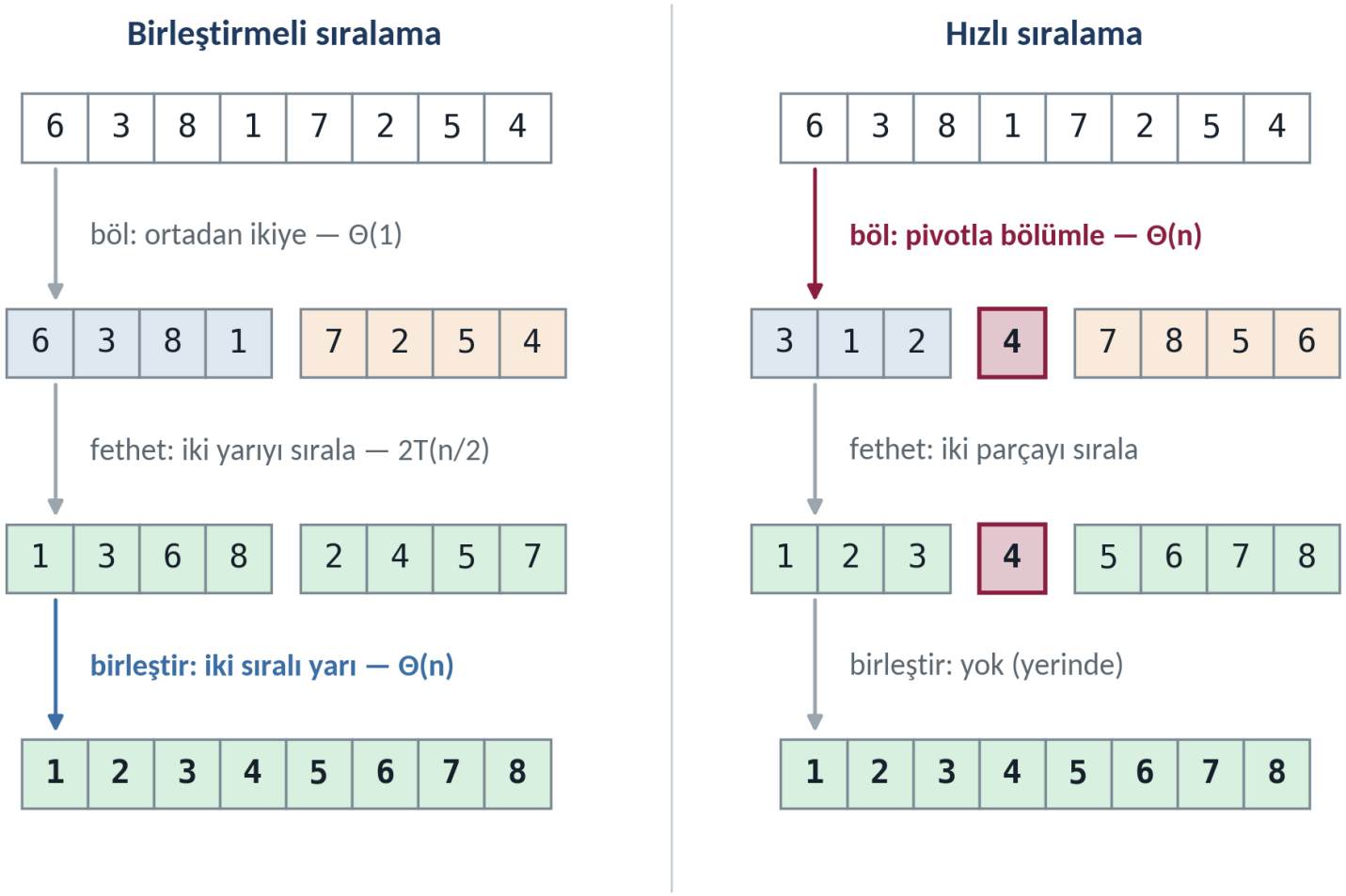
Shustek (2009); Hoare (1981, 1961a, 1961b, 1962); Knuth (1998); McIlroy (1999); Spicer (2026): Hoare 5 Mart 2026'da hayatını kaybetti.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 6.1	Böl ve fethet stratejisini üç adımda (böl, fethet, birleştir) ifade eder ve problemlere uygular.
ÖÇ 6.2	İki sıralı diziyi doğrusal zamanda birleştirir; birleştirmeli sıralamayı gerçekler ve O(n log n) karmaşıklığını özyineleme ağacıyla gösterir.
ÖÇ 6.3	Lomuto ve Hoare bölümleme yöntemleriyle hızlı sıralamayı gerçekler; pivot seçiminin en iyi, en kötü ve ortalama duruma etkisini açıklar.
ÖÇ 6.4	Böl ve fethet yineleme bağıntılarını özyineleme ağacı ve basitleştirilmiş ana teoremle çözer.
ÖÇ 6.5	Karşılaştırmaya dayalı sıralamanın Ω(n log n) alt sınırını karar ağacıyla açıklar; quickselect ile k'ıncı en küçük elemanı ortalama doğrusal zamanda bulur.

Böl – fethet – birleştir: iş nerede?



- **Böl:** aynı türden küçük alt problemler.
- **Fethet:** özyinelemeyle çöz; küçük olanı doğrudan.
- **Birleştir:** alt çözümlerden asıl çözüm.
- Birleştirmeli: iş **birleştirmede**; hızlı: iş **bölmede** (bölümleme).
- İkili arama: tek yarı, birleştirme yok.

Yineleme bağıntısı ve yerine koyma

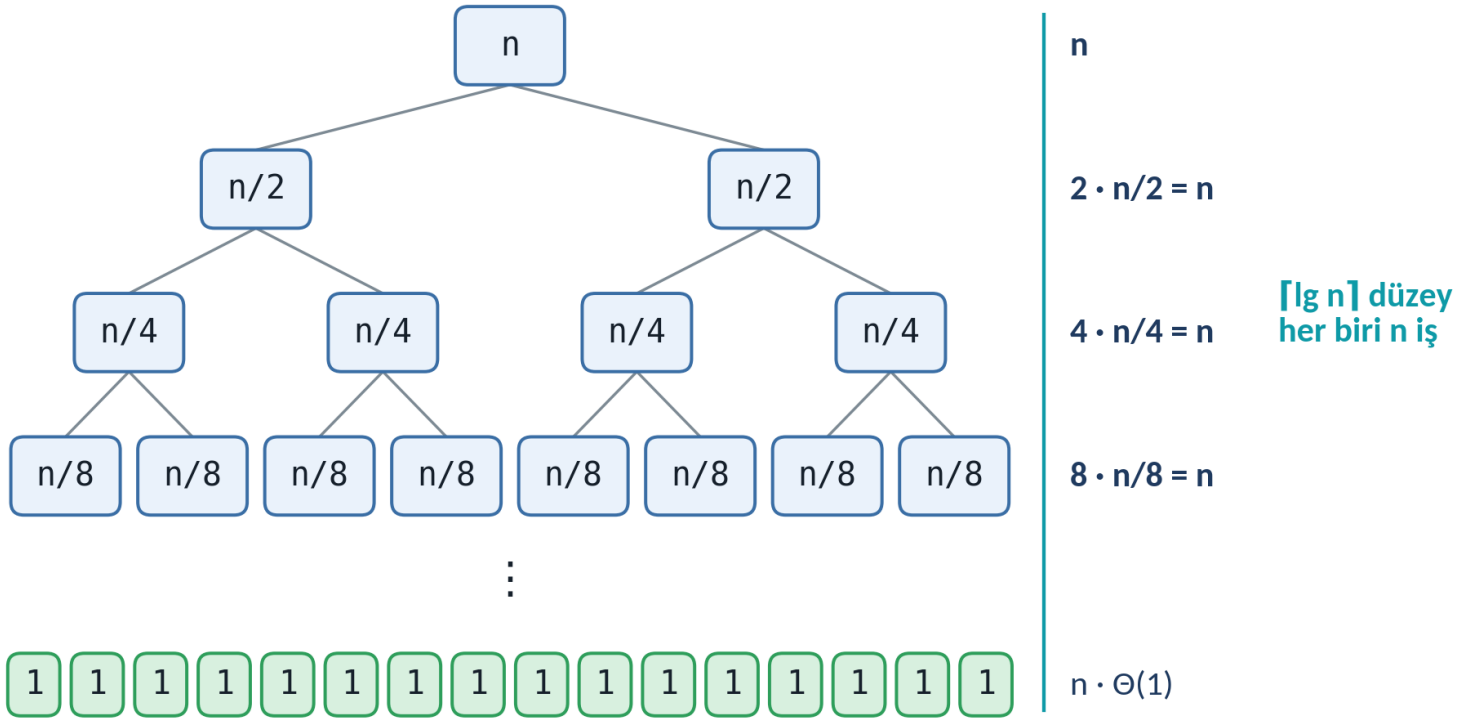
- $T(n) = 2T(n/2) + n = 4T(n/4) + 2n = \ldots = 2^kT(n/2^k) + k{\cdot}n \rightarrow$ **$n + n \lg n$**
- $T(n) = T(n-1) + n \rightarrow$ **$n(n+1)/2$** (pivot hep uçta)
- $T(n) = T(n/2) + 1 \rightarrow$ **$\lg n + 1$** (ikili arama)
- $T(n) = 2T(n/2) + 1 \rightarrow$ **$2n - 1$**
- Kavram 2 dört bağıntıyı **özyinelemeyle hesaplar** ve kapalı biçimle karşılaştırır (aşağıda).

ÇIKTI · KAVRAM 2 (SEÇİLİ SATIRLAR)

n = 2^k	2T(n/2)+n	n·k+n	T(n-1)+n	n(n+1)/2	T(n/2)+1	k+1	2T(n/2)+1	2n-1
1	1	1	1	1	1	1	1	1
8	32	32	36	36	4	4	15	15
64	448	448	2080	2080	7	7	127	127
1024	11264	11264	524800	524800	11	11	2047	2047
4096	53248	53248	8390656	8390656	13	13	8191	8191

kapalı biçimler bağıntılarla aynı

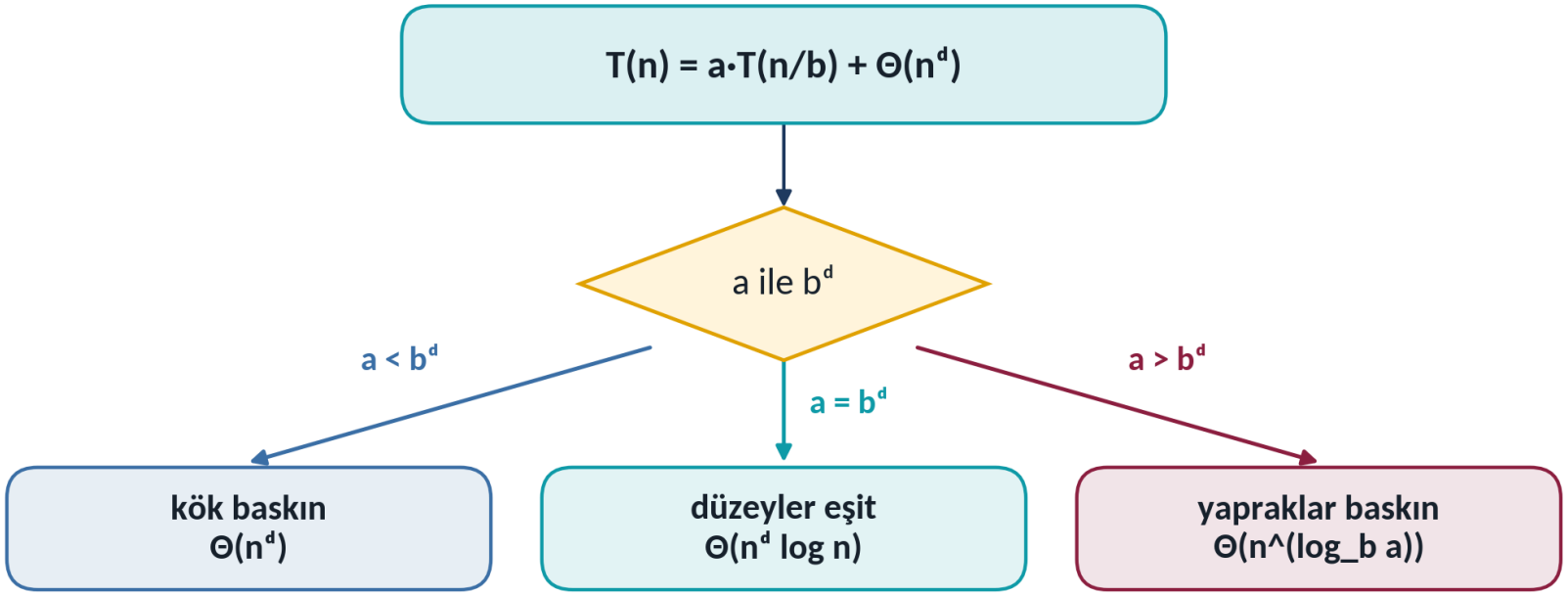
Özyineleme ağacı: her düzeyde n iş, ⌈lg n⌉ düzey



- d. düzeyde 2^d düğüm × $n/2^d$ iş = **n**.
- ⌈lg n⌉ iş yapan düzey → **n·⌈lg n⌉**.
- Örnek 3, n = 1024: her düzeyde 1024 iş; toplam iş = 10240.
- Karşılaştırma işten biraz az: kalan karşılaştırmasız kopyalanır.

T(n) = 2T(n/2) + n → toplam ≈ n·lg n (Örnek 3: n = 1024 için iş yapan her düzeyde 1024, toplam 10240)

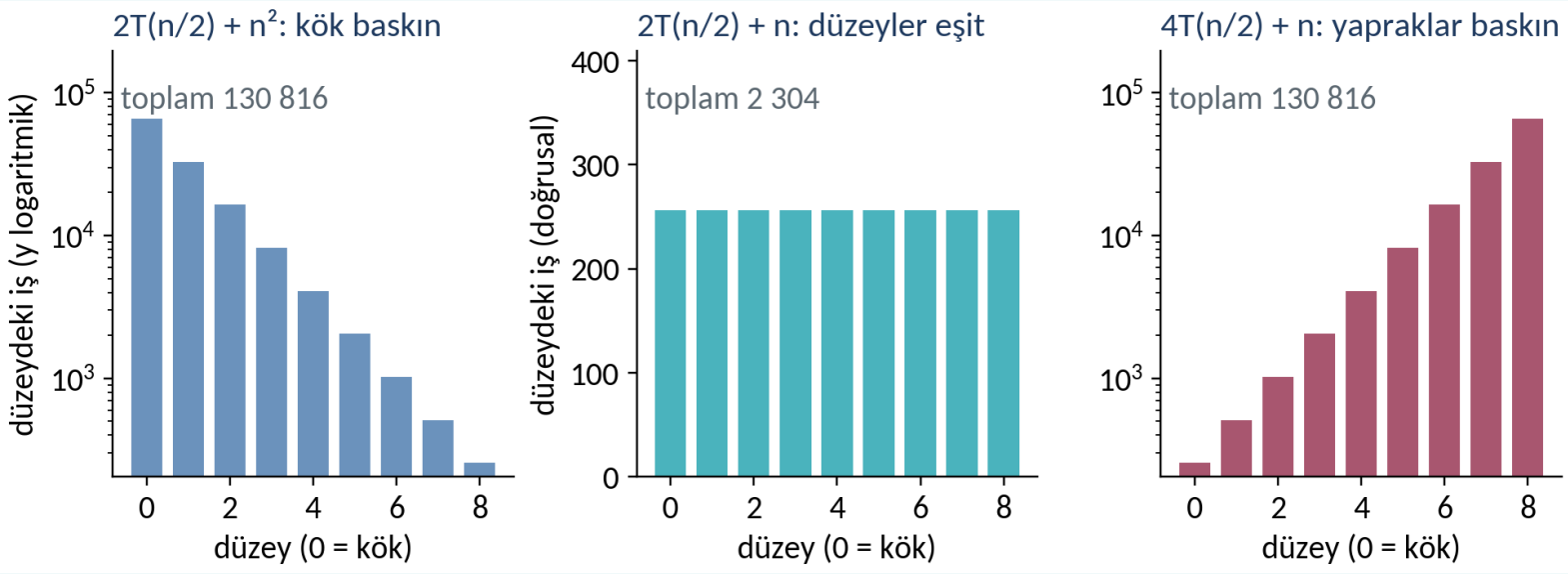
Ana teorem (basitleştirilmiş): a ile b^d'yi karşılaştır



- **$T(n) = a \cdot T(n/b) + \Theta(n^d)$**
- Düzey işleri **a/b^d** oranlı geometrik dizi.
- $a < b^d$: kök baskın → $\Theta(n^d)$
- $a = b^d$: eşit → $\Theta(n^d \log n)$
- $a > b^d$: yapraklar → $\Theta(n^{\log_b a})$
- Kalıp dışı: $T(n-1) + n$, $T(n/3) + T(2n/3) + n$.

TAHMİN ET

T(n) = 3T(n/2) + n hangi duruma girer?



A — Yapraklar baskın: Θ(n^(log₂ 3)) ≈ Θ(n^1,585)

B — Düzeyler eşit: Θ(n log n)

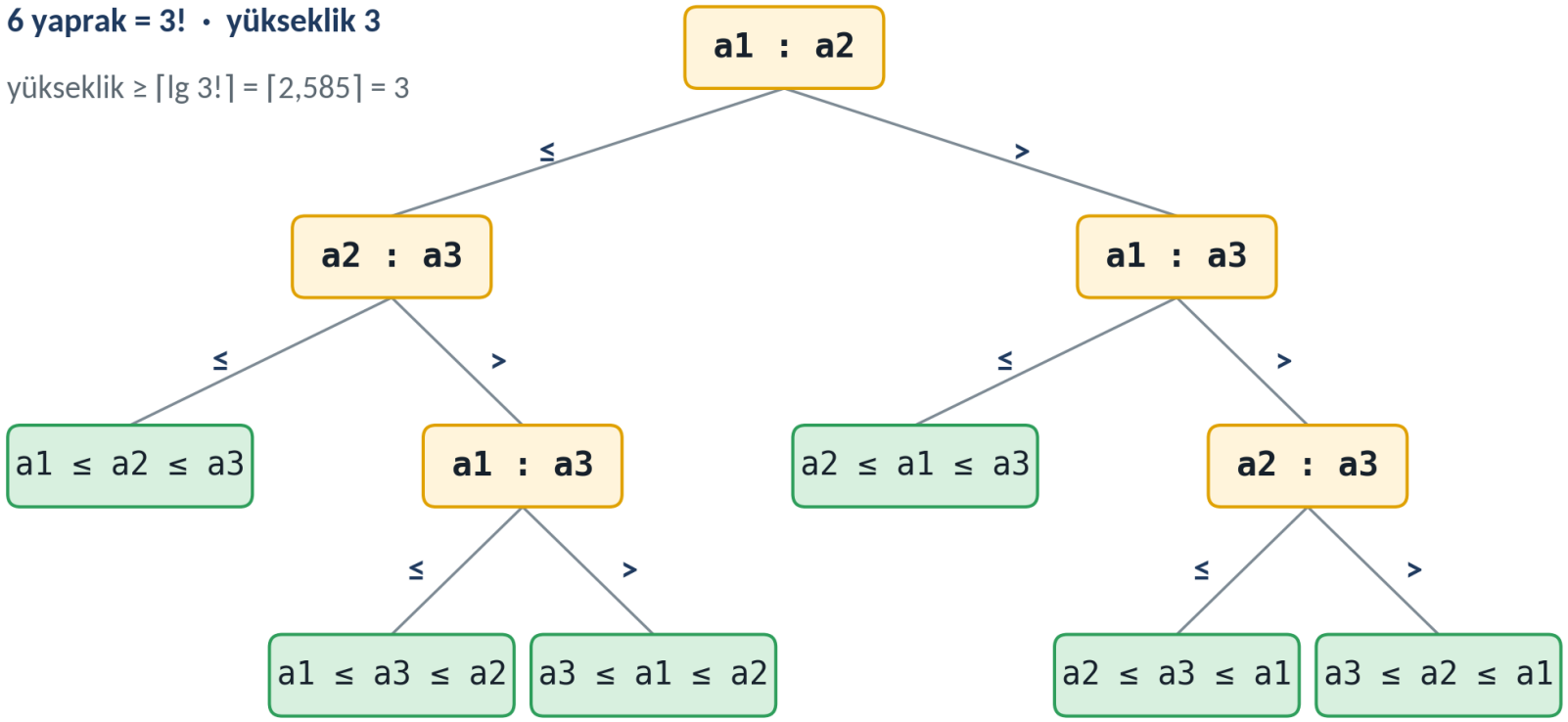
C — Kök baskın: Θ(n)

D — Θ(n²): üç alt problem karesel verir

Karar ağacı: neden n log n'den hızlısı yok?

6 yaprak = 3! · yükseklik 3

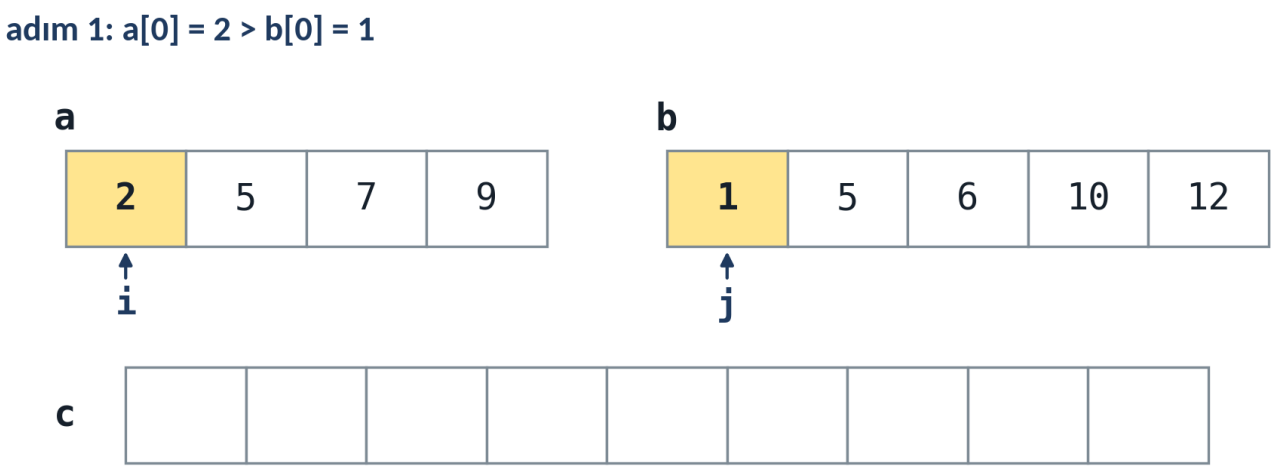
yükseklik ≥ ⌈lg 3!⌉ = ⌈2,585⌉ = 3



her iç düğüm bir karşılaştırma; her yaprak bir sıralama (permütasyon); a1 = a[0]

- Karşılaştırmaya dayalı sıralama = ikili **karar ağacı**.
- n! sıralamanın her biri bir yaprak; h yüksekliğinde ≤ 2^h yaprak.
- $2^h \geq n!$ → **$h \geq \lg(n!) = \Theta(n \log n)$**.
- n = 8: ⌈lg 8!⌉ = 16; birleştirmeli en kötü W(8) = 17.
- H7: karşılaştırmasız sıralamalar bu sınırın dışında.

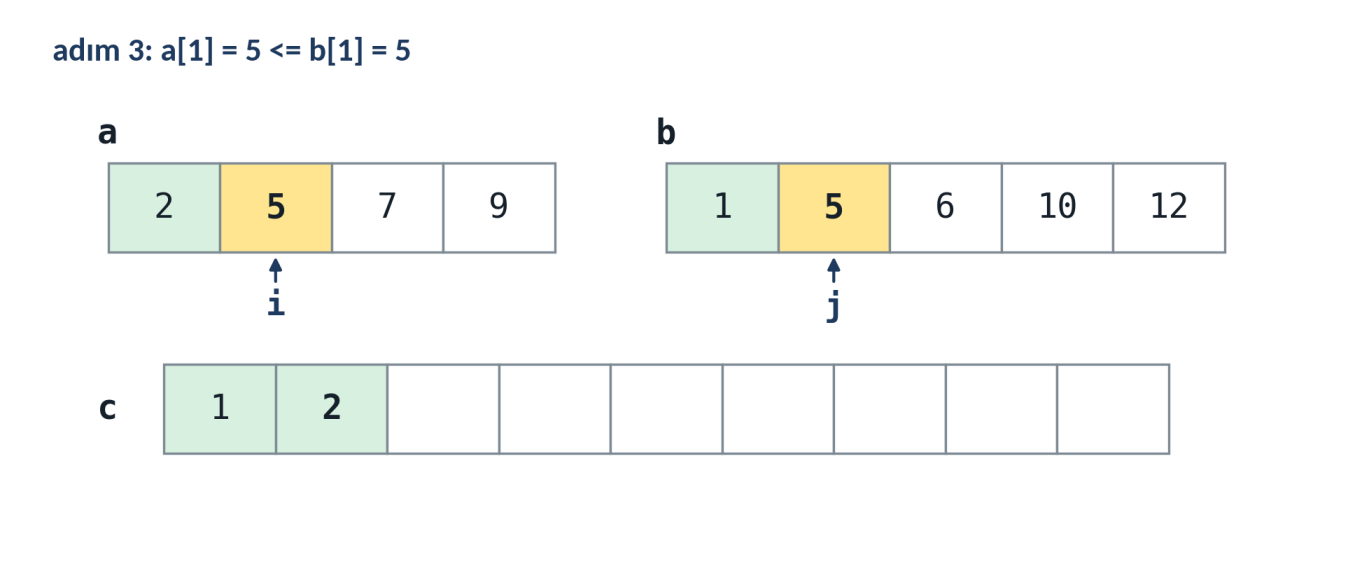
Birleştirme: iki işaretçi, küçüğü al



- **Problem:** iki sıralı dizi → tek sıralı dizi.
- a[i] ile b[j] karşılaştırılır; **küçük olan** c'ye, işaretçisi ilerler.
- İlk adım: a[0] = 2 > b[0] = 1 → b'den 1.
- Her adımda bir eleman yerleşir: iş **Θ(na + nb)**.

TAHMİN ET

Adım 3: a[1] = 5, b[1] = 5. Hangisi önce c'ye yazılır?



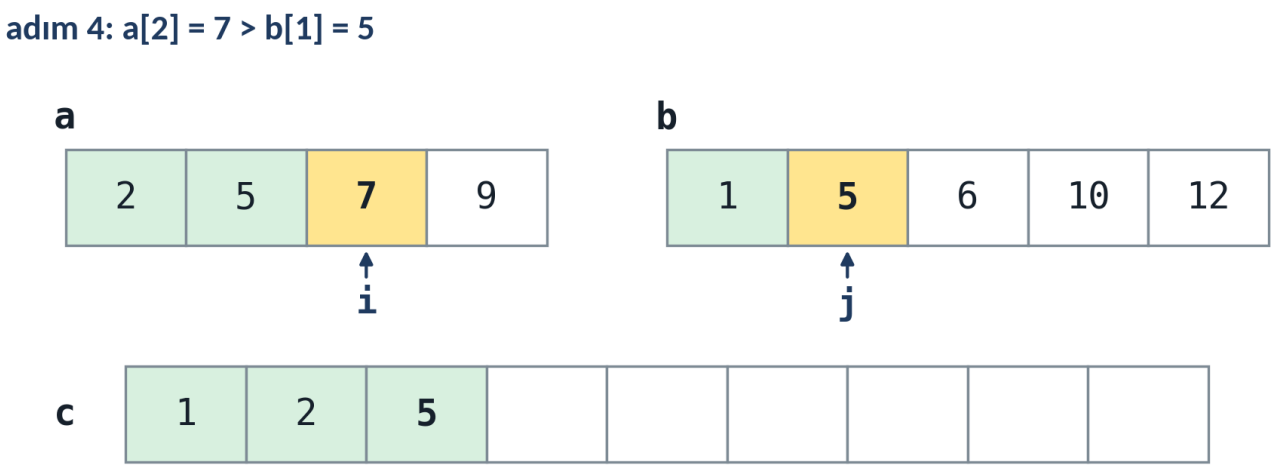
A b'deki 5 (sağdaki dizi önceliklidir)

B a'daki 5 (eşitlikte soldan al: <=)

C İkisi aynı anda yazılır

D Fark etmez; sonuç her durumda aynıdır

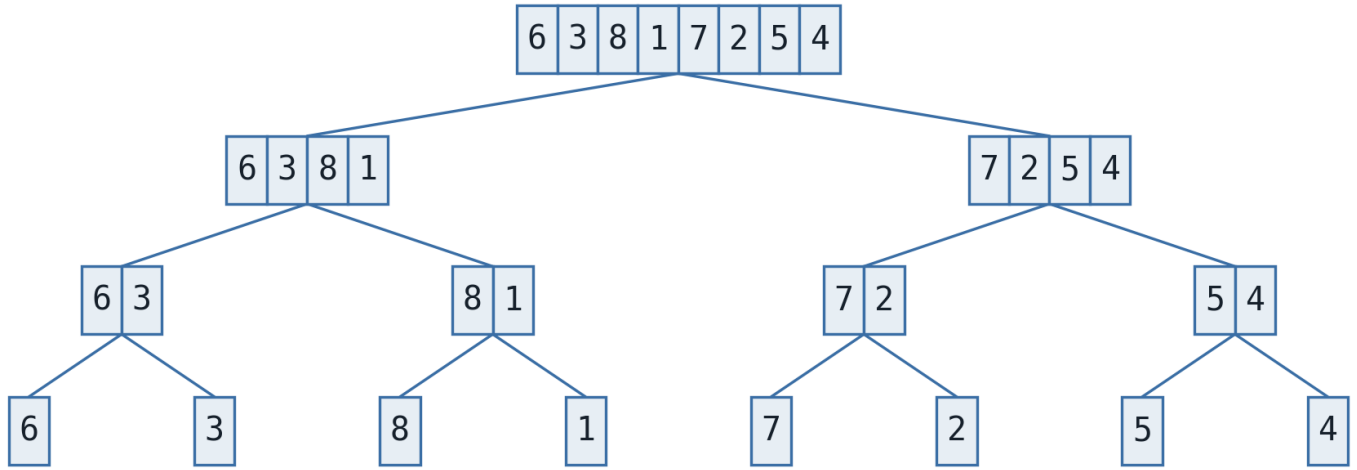
Eşitlikte soldan: kararlılık bu tek karakterde



- Yanıt: **a'daki 5** alındı (<=).
- < yazılsaydı b'deki 5 önce gelirdi: eşit anahtarlı kayıtların sırası bozulur (**HA6**).
- Sonra: a biter, b'nin kalanı [10, 12] karşılaştırmasız kopyalanır (unutmak: HA1).
- Toplam 7 karşılaştırma ≤ na + nb − 1 = 8.
- **Atölye 1:** iki sıralı 8'lik kart destesini birleştirin, karşılaştırmaları sayın.

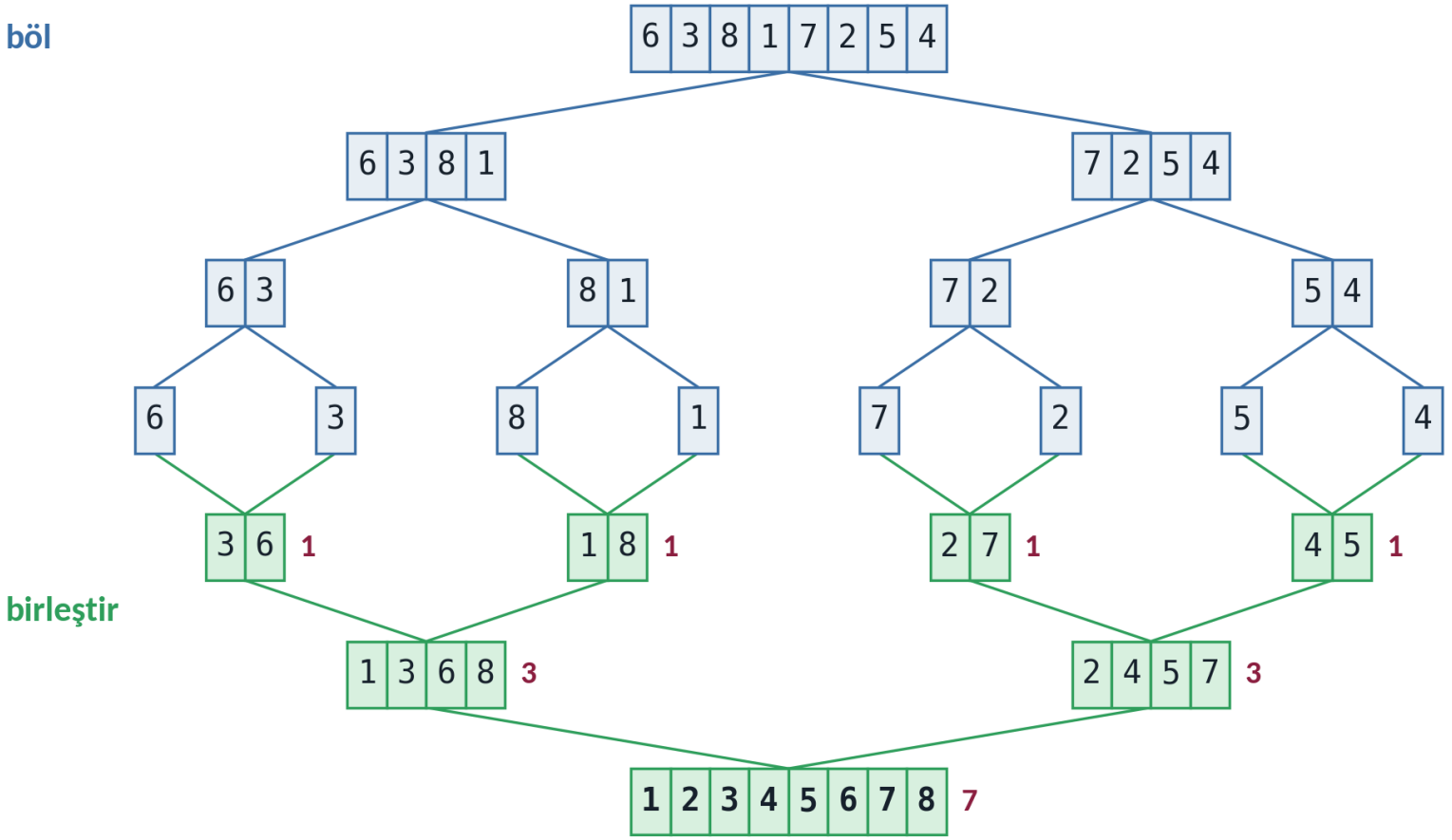
Birleştirmeli sıralama: önce böl …

böl



- Ortadan ikiye: **mid = lo + ⌊(hi − lo)/2⌋**; yukarı yuvarlamak [0..1]'i küçültmez (**HA4**).
- Tek elemanlı parça zaten sıralıdır (temel durum).
- Düzey sayısı ⌈lg n⌉ + 1 = 4.
- **Tahmin:** kök birleştirmesi kaç karşılaştırma yapar?

… sonra birleştir: düzey başına en çok n karşılaştırma



kırmızı sayılar: o birleştirmedeki karşılaştırma; toplam 17 (Örnek 2)

BİRLEŞTİR: sözde kod ve C

```
1  BİRLEŞTİR(a, lo, mid, hi)
2    i ← lo; j ← mid + 1; k ← lo
3    i ≤ mid ve j ≤ hi olduğu sürece
4      eğer a[i] ≤ a[j] ise
5        yrd[k++] ← a[i++]
6      değilse yrd[k++] ← a[j++]
7    kalanları kopyala (iki döngü)
8    a[lo..hi] ← yrd[lo..hi]
```

c/H06_02_birlestirmeli_siralama.c · satır 24–38 — C

```c
24      int i = lo, j = mid + 1, k = lo;
25      while (i <= mid && j <= hi) {
26          karsilastirma++;
27          if (a[i] <= a[j]) {  /* eşitse soldan */
28              yrd[k++] = a[i++];
29          } else {
30              yrd[k++] = a[j++];
31          }
32      }
33      while (i <= mid) {
34          yrd[k++] = a[i++];
35      }
36      while (j <= hi) {
37          yrd[k++] = a[j++];
38      }
```

Değişmez: yrd[lo..k−1] sıralı ve a[lo..i−1] ile a[mid+1..j−1]'in tamamını içerir. Geri kopyalama satır 39–41.

BİRLEŞTİRMELİ-SIRALA: böl, fethet, birleştir

```
1  BS(a, lo, hi, d)
2    eğer lo ≥ hi ise döndür
3    mid ← lo + ⌊(hi − lo)/2⌋
4    BS(a, lo, mid, d + 1)
5    BS(a, mid + 1, hi, d + 1)
6    BİRLEŞTİR(a, lo, mid, hi)
```

c/H06_02_birlestirmeli_siralama.c · satır 48–60 — C

```c
48      cagri++;
49      if (d > en_derin) {
50          en_derin = d;
51      }
52      if (lo >= hi) {  /* 0 ya da 1 eleman: sıralı */
53          return;
54      }
55      int mid = lo + (hi - lo) / 2;
56      /* böl ve fethet: iki yarıyı ayrı ayrı sırala */
57      bs_ozyineli(a, yrd, lo, mid, d + 1);
58      bs_ozyineli(a, yrd, mid + 1, hi, d + 1);
59      birlestir(a, yrd, lo, mid, hi);  /* birleştir */
60  }
```

`bs_ozyineli(a, yrd, lo, hi, d)` (satır 46) · doğruluk tümevarımla · T(n) = 2T(n/2) + n → Θ(n log n), her girdide.

Tek tampon: bir kez ayır, bütün çağrılarla paylaş

c/H06_02_birlestirmeli_siralama.c · satır 63–76

C

```c
63  void birlestirmeli_sirala(int *a, size_t n)
64  {
65      if (n == 0) {
66          return;
67      }
68      int *yrd = malloc(n * sizeof *yrd);
69      if (yrd == NULL) {
70          fprintf(stderr, "bellek yetersiz\n");
71          exit(1);
72      }
73      tampon_ayirma++;
74      bs_ozyineli(a, yrd, 0, (int)n - 1, 1);
75      free(yrd);
76  }
```

ÇIKTI · ÖRNEK 2 (8 ELEMAN)

```
karşılaştırma = 17 · kopya = 48 · çağrı = 15 · derinlik = 4
tampon ayırma = 1 (her çağrıda değil, bir kez)
```

ALGORİTMA KİMLİK KARTI

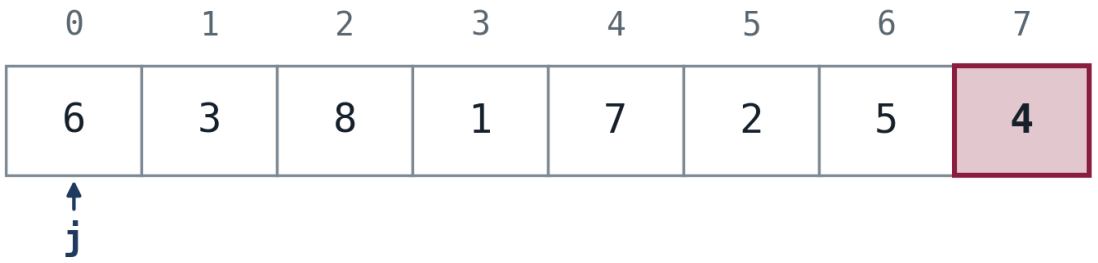
Birleştirmeli sıralama *merge sort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Böl ve fethet: ortadan böl, sırala, doğrusal birleştir	Yok; n elemanlık ek tampon	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(n) tampon + O(log n) yığın

Özellikler	Kararlı (eşitlikte soldan al: <=) · yerinde değil · uyarlanabilir değil (sıralı girdide bile (n/2)·lg n karşılaştırma).
Değişmez / doğruluk	birlestir: yrd[lo..k−1] sıralıdır ve a[lo..i−1] ile a[mid+1..j−1]'in tamamını içerir. bs(lo, hi) döndüğünde a[lo..hi] sıralıdır
Kütüphane karşılığı	C: yok (elle); `qsort` algoritma ve kararlılık garantisi vermez · C++: `std::stable_sort` (kararlı), `std::merge`, `std::inplace_merge` · Python: `sorted(a)`, `a.sort()` (Timsort: birleştirmeli + araya ekleme melezi, kararlı); `heapq.merge`
Ne zaman kullanılır	Kararlılık gerektiğinde (çok ölçütlü sıralama), en kötü durum Θ(n log n) garantisi istendiğinde.
Ne zaman kullanılmaz	Ek bellek kısıtlıysa (O(n) tampon); çok küçük dizilerde (araya ekleme daha hızlı: melez sürüm)

Hızlı sıralama: pivot seç, bölümle

başta: i = −1, j = 0



- **Fikir:** pivottan küçükler sola, diğerleri sağa; pivot araya, **son yerine**.
- Birleştirmeye gerek yok: iş **bölmede**.
- **Lomuto:** pivot = a[hi] = 4; i = −1, j soldan sağa.
- a[j] < pivot → i++ ve a[i] ↔ a[j].

TAHMİN ET

j = 3: a[3] = 1 < pivot. Şimdi ne olur?

j = 2: a[j] = 8 ≥ 4 → yalnız j ilerler



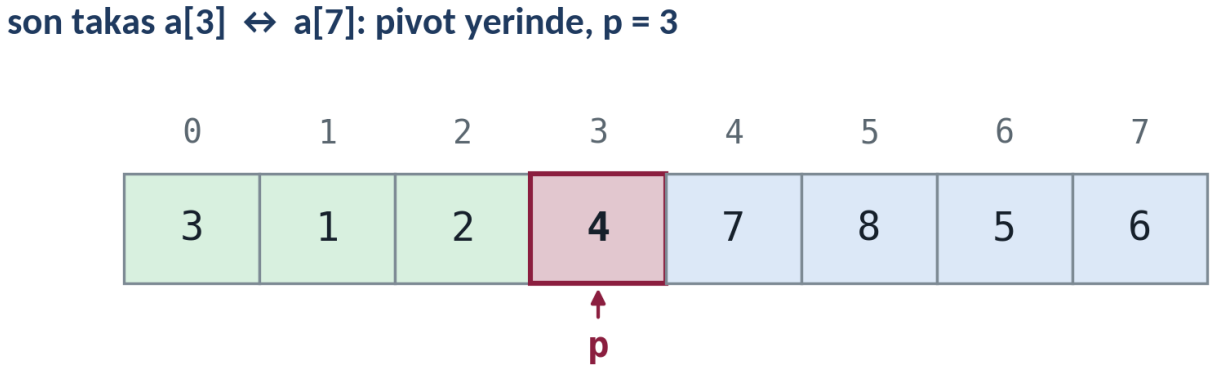
A Takas yok; yalnızca j ilerler

B a[0] ↔ a[3]; i = 0 kalır

C i = 1; a[1] ↔ a[3] (küçükler bölgesi büyür)

D Pivot hemen a[3]'e taşınır

Son takas: pivot yerinde, p = 3



- j = 3'te i = 1 oldu ve a[1] ↔ a[3] (önceki sorunun yanıtı).
- Döngü bitti: a[i + 1] ↔ a[hi] → **p = 3**.
- 7 karşılaştırma, 4 takas (Örnek 4).
- Sol [3, 1, 2] ve sağ [7, 8, 5, 6] **bağımsız** sıralanır.

LOMUTO-BÖLÜMLE: sözde kod ve C

```
1  LOMUTO(a, lo, hi)
2    pivot ← a[hi]; i ← lo − 1
3    j ← lo … hi − 1 için
4      eğer a[j] < pivot ise
5        i ← i + 1; a[i] ⇔ a[j]
6    a[i + 1] ⇔ a[hi]
7    döndür i + 1
```

c/H06_04_lomuto_hizli.c · satır 28–41

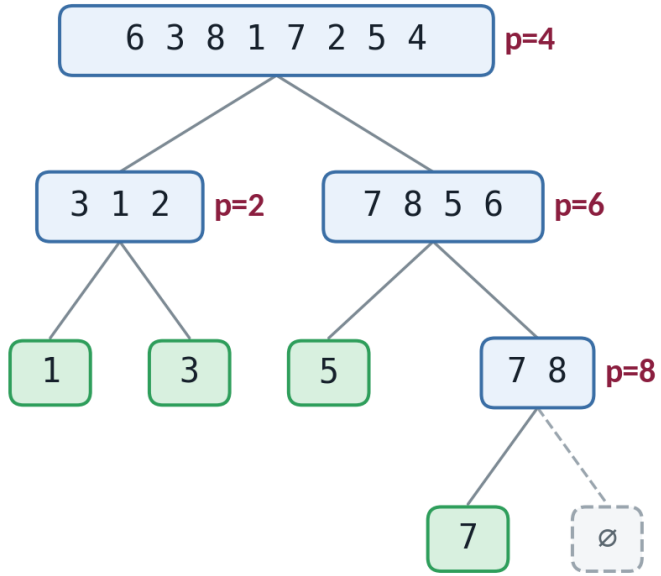
C

```c
28  int lomuto_bolumle(int *a, int lo, int hi)
29  {
30      int pivot = a[hi];
31      int i = lo - 1;
32      for (int j = lo; j < hi; j++) {
33          karsilastirma++;
34          if (a[j] < pivot) {
35              i++;
36              takas(a, i, j);
37          }
38      }
39      takas(a, i + 1, hi);  /* pivot son yerine */
40      return i + 1;
41  }
```

Değişmez: a[lo..i] < pivot ≤ a[i+1..j−1], a[j..hi−1] işlenmemiş. HIZLI(lo, p − 1) ve HIZLI(p + 1, hi); birleştirme yok.

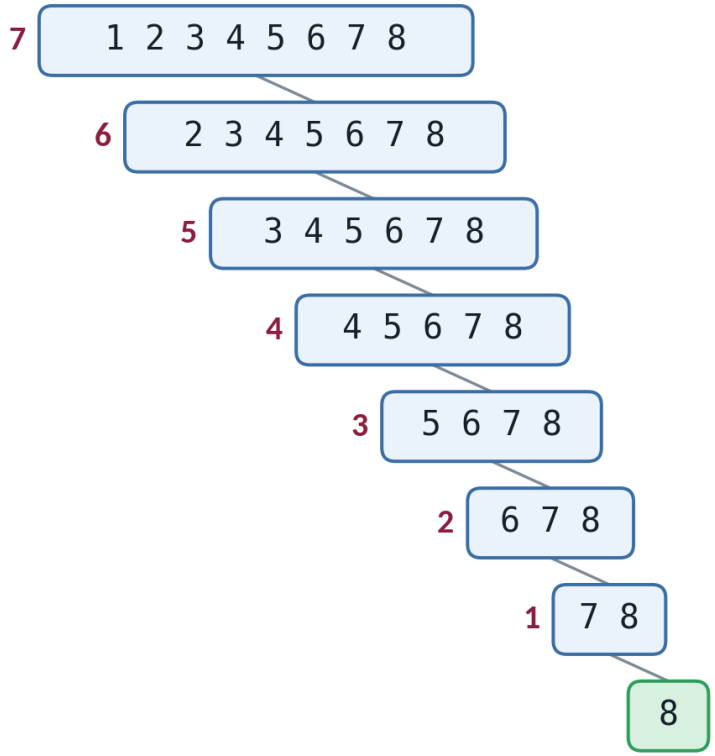
En iyi ve en kötü: dengeli ağaç ↔ zincir

(a) haftanın dizisi: derinlik 4, 13 karşılaştırma



p: pivot (Lomuto, a[hi]); yeşil: tek elemanlı parça

(b) sıralı 1..8, ilk eleman pivot: derinlik 8



her düzeyde n−1 | 0 bölünme (boş parçalar çizilmedi)

kırmızı: karşılaştırma; toplam 28 = 8·7/2

Hoare bölümleme: iki uçtan tara

başta: x = a[lo] = 6; i = lo − 1, j = hi + 1

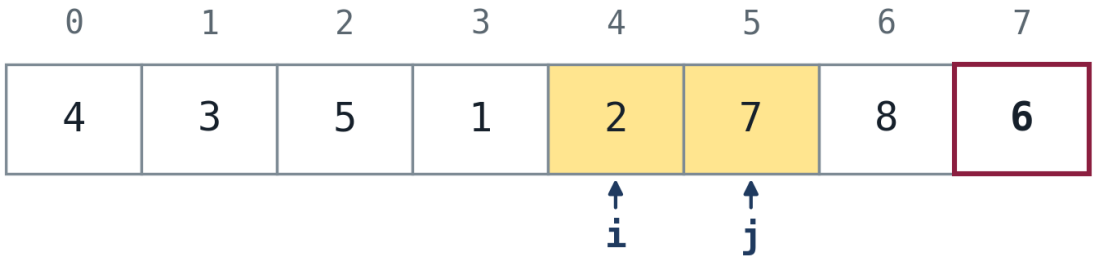
0	1	2	3	4	5	6	7
6	3	8	1	7	2	5	4

- Pivot **x = a[lo] = 6**; i = lo − 1, j = hi + 1.
- j sağdan x'ten **büyük olmayanda**, i soldan **küçük olmayanda** durur.
- i < j ise a[i] ↔ a[j]; i ≥ j ise **j döndürülür**.
- Takası yalnızca iki eleman da yanlış taraftaysa yapar.

TAHMİN ET

3 takastan sonra tur 4: i ≥ j olunca ne döner, pivot 6 nerededir?

tur 3: j sağdan 5'te, i soldan 4'te durur → takas



A j = 5, i = 5; pivot a[4]'te

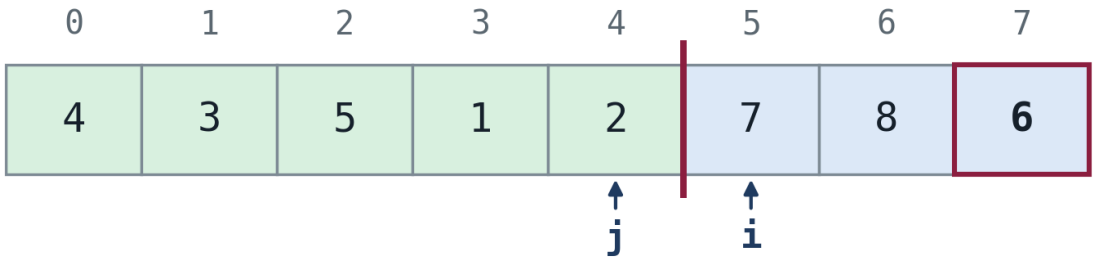
B j = 4, i = 5; dönüş 4; pivot a[4]'te

C j = 4, i = 4; dönüş 4; pivot a[0]'da

D j = 4, i = 5; dönüş 4; pivot a[7]'de

Dönüş j = 4: sınır, pivotun yeri değil

tur 4: j = 4, i = 5; i ≥ j → dönüş j = 4



- a[0..4] ≤ 6 ≤ a[5..7]; pivot a[7]'de.
- Özyineleme **[lo..j] ve [j+1..hi]** (Lomuto'da [lo..p−1], [p+1..hi]).
- 10 karşılaştırma, 3 takas; Lomuto aynı dizide 7 ve 4.
- Tümü eşit girdide Hoare ortadan böler, Lomuto karesel çöker (Örnek 5).

Pivot seçimi: ilk eleman / rastgele / üçün ortancası

	ilk eleman	rastgele	üçün ortancası
sıralı	**91,15** ağaç 2000	**1,25** ağaç 26	**0,91** ağaç 11
ters	**91,15** ağaç 2000	**1,12** ağaç 27	**1,67** ağaç 24
rastgele	**1,13** ağaç 25	**1,13** ağaç 30	**1,06** ağaç 17

karşılaştırma / (n·lg n) n = 2000

karesel: n(n−1)/2 / (n lg n) ≈ 91,1 · 2(n+1)H(n) − 4n ≈ 1,39 n lg n · kaynak: Örnek 6'nın fonksiyonları

- Hücre: karşılaştırma / (n lg n), n = 2000.
- Sıralı ve ters girdide **ilk eleman** karesel çöker.
- **Rastgele** pivot: hiçbir sabit girdi en kötüyü tetikleyemez.
- **Üçün ortancası:** sıralı girdide mükemmel; ters girdide daha zayıf.
- Küçük parçaya özyineleme: yığın ≤ ⌊lg n⌋ + 1 (ağaç 1000 iken yığın 2).

TAHMİN ET

Sıralı 1000 elemanda ilk eleman pivotu kaç karşılaştırma yapar?

A ≈ 10 757 (rastgele girdideki kadar)

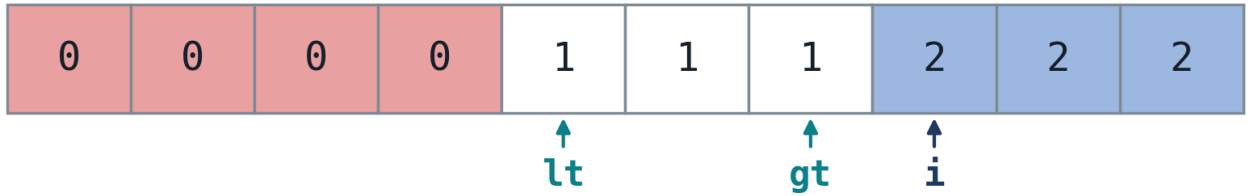
B 499 500 = n(n − 1)/2

C 999 = n − 1 (tek bölümleme yeter)

D 9 009 (n lg n'den az)

Eşit anahtarlar: üç yollu bölümleme (Hollanda bayrağı)

adım 10: i > gt → dur



- Üç bölge: **< p | = p | > p**; işaretçiler lt, i, gt.
- a[i] < p: a[lt] ↔ a[i], lt++, i++.
- a[i] > p: a[i] ↔ a[gt], **yalnız gt−−**.
- a[i] = p: i++.
- Tek geçiş: 16 karşılaştırma, 7 takas. 0/1 girdide (n = 2000) iki yollu 1 001 099, üç yollu 5 980 karşılaştırma.

ALGORİTMA KİMLİK KARTI

Hızlı sıralama *quicksort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Böl ve fethet: pivotla bölümle, iki parçayı sırala	Yok	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ yığın (küçük parçaya özyinelemeyle)

Özellikler	Kararsız · yerinde · pivot seçimine duyarlı: sıralı girdide ilk eleman pivotu n(n−1)/2 karşılaştırma ve n derinlik verir.
Değişmez / doğruluk	Lomuto: a[lo..i] < pivot ≤ a[i+1..j−1], a[j..hi−1] işlenmemiş, a[hi] = pivot. Bölümleme sonunda pivot son yerindedir
Kütüphane karşılığı	C: `qsort` (algoritma standartta belirtilmez) · C++: `std::sort` (libstdc++: introsort, kararsız); `std::partition` · Python: yok (`list.sort` Timsort'tur, hızlı sıralama değil)
Ne zaman kullanılır	Genel amaçlı bellek içi sıralama; ek bellek istenmediğinde; ortalama hızın önemli olduğu yerlerde (sıralı bellek erişimi, önbellek dostu bölümleme)
Ne zaman kullanılmaz	Kararlılık gerektiğinde; en kötü durum garantisi şartsa ya da girdi saldırgan olabilirse (McIlroy, 1999) — introsort ya da birleştirmeli sıralama

Quickselect: yalnızca k'yı içeren parçaya devam

k = 4 (0 tabanlı): 9 notun ortancası; seçilen parçada devam, ötekiler elenir

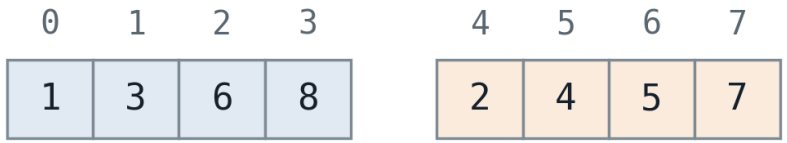


lo = hi = 4: bulundu = 72 (Örnek 9) · sarı: pivota eşitler (üç yollu bölümleme) · gri: elenen

- 9 notun ortancası (k = 4): 3 turda **72**.
- Üç yollu bölümleme; k ∈ [lt..gt] ise yanıt pivot.
- T(n) ≈ T(n/2) + n → **Θ(n)** (ana teorem: a = 1 < b^d).
- Karşılaştırma/n, n 1000 kat büyürken: 4,24, 4,90, 3,89, 4,74.
- En kötü Θ(n^2); C++: `std::nth_element`.

Ters çevrim sayısı: birleştirirken say

kök birleştirmesi: sol a[0..3] ve sağ a[4..7] (ikisi de sıralı)




```
sağdan 2 (i = 1): +3 (3,2)(6,2)(8,2)
sağdan 4 (i = 2): +2 (6,4)(8,4)
sağdan 5 (i = 2): +2 (6,5)(8,5)
sağdan 7 (i = 3): +1 (8,7)
```

kesişen ters çevrim = 8

sayi += mid − i + 1: a[i..mid] sıralı ve hepsi a[j]'den büyük,

yani sağdan alınan a[j] soldaki her kalan elemanla bir çift oluşturur

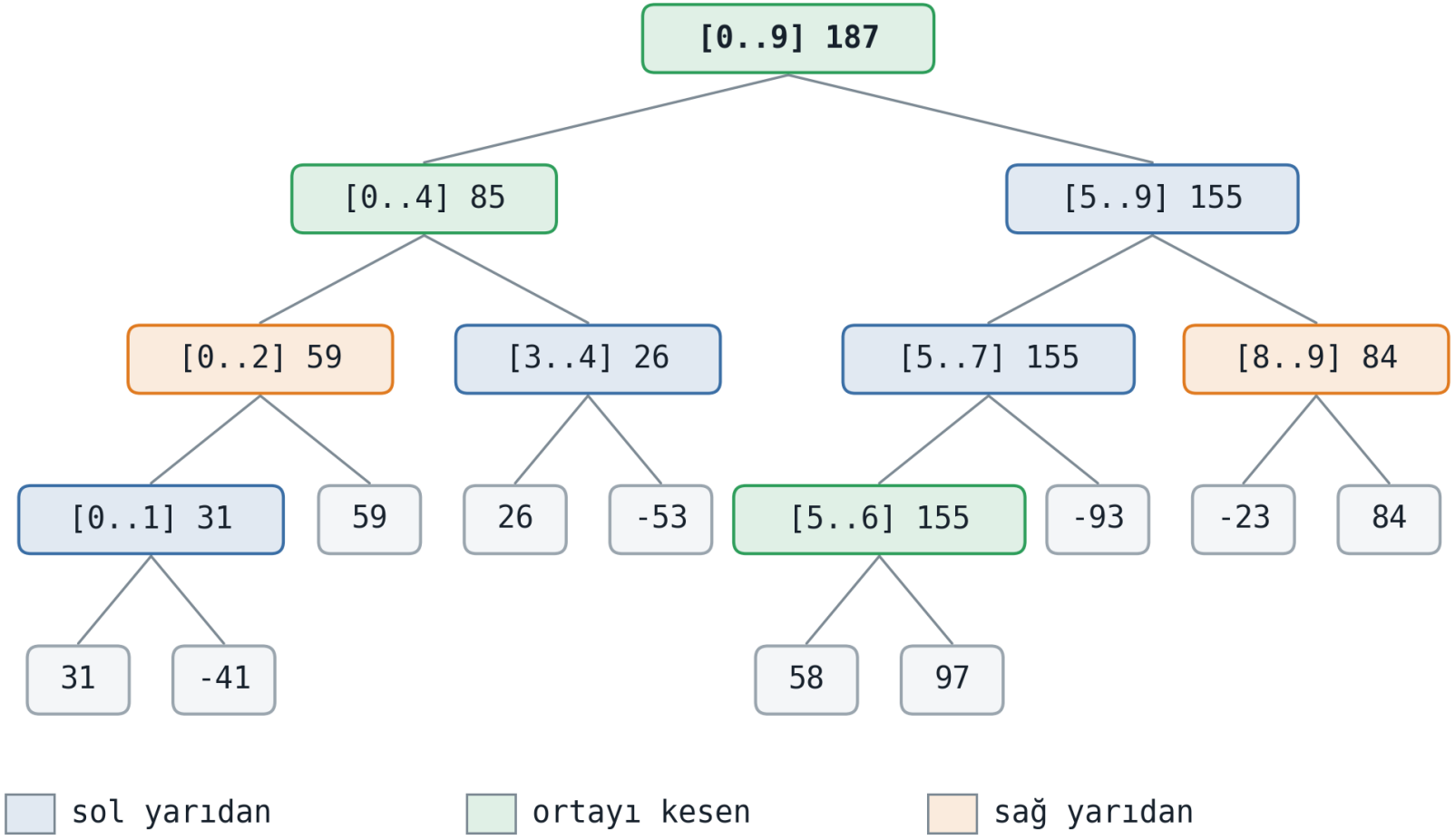
toplam = sol yarıdaki + sağ yarıdaki + kesişen çiftler

haftanın dizisinde 16 ters çevrim (Örnek 8; $\Theta(n^2)$ kaba yöntemle aynı)

- Sağdan a[j] alınınca solda kalan **mid − i + 1** elemanın her biri a[j]'den büyük.
- Toplam = sol + sağ + kesişen; $\Theta(n \log n)$.
- n = 100 000: 2 494 795 927 ters çevrim 1 536 273 karşılaştırmayla.
- Kâhin: H5'in $\Theta(n^2)$ çift taraması.
- Sayaç `long long`: ters sıralıda $n(n-1)/2$.

En büyük alt dizi: sol, sağ ya da ortayı kesen

a = 31 -41 59 26 -53 58 97 -93 -23 84 → en büyük toplam 187 = a[2..6]



- Bentley'nin dizisi: sol 85, sağ 155, kesişen **187** = a[2..6].
- Kesişen: mid'den sola + mid + 1'den sağa en iyi toplam, Θ(n).
- T(n) = 2T(n/2) + n → Θ(n log n).
- n = 1000: 8 977 toplama (= W(n)); Kadane 999.
- Kâhin: H2'nin Kadane'si.

Üç Dilde: birleştirme döngüsü (Örnek 10, aşağıdan yukarıya)

C

```c
while (i <= mid && j <= hi) {
    karsilastirma++;
    if (a[i] <= a[j]) {
        yrd[k++] = a[i++];
    } else {
        yrd[k++] = a[j++];
    }
}
```

C++

```cpp
while (i <= mid && j <= hi) {
    karsilastirma++;
    if (a[i] <= a[j]) {
        yrd[k++] = a[i++];
    } else {
        yrd[k++] = a[j++];
    }
}
```

Python

```python
while i <= mid and j <= hi:
    karsilastirma += 1
    if a[i] <= a[j]:
        yrd[k] = a[i]
        i += 1
    else:
        yrd[k] = a[j]
        j += 1
    k += 1
```

C ve C++'ta `yrd[k++] = a[i++]` tek satır; Python'da `++` yok: artırmalar ayrı satırlarda (hardal). Eşitlikte soldan (`<=`) üç dilde aynı. (H06_10_asagidan_yukari.c · .cpp · .py)

Üç Dilde: iki değer döndürmek (üç yollu bölümleme)

H06_07_uc_yollu_bolumleme.c — C

```c
int p = a[rastgele_aralik(lo, hi)];
int lt, gt;
uc_yollu_bolumle(a, lo, hi, p, &lt, &gt);
/* a[lt..gt] = p: eşitler bir daha işlenmez */
hizli_uc_yollu(a, lo, lt - 1, d + 1);
hizli_uc_yollu(a, gt + 1, hi, d + 1);
```

H06_07_uc_yollu_bolumleme.cpp — C++

```cpp
int p = a[rastgele_aralik(lo, hi)];
auto [lt, gt] = uc_yollu_bolumle(a, lo, hi, p);
// a[lt..gt] = p: eşitler bir daha işlenmez
hizli_uc_yollu(a, lo, lt - 1, d + 1);
hizli_uc_yollu(a, gt + 1, hi, d + 1);
```

H06_07_uc_yollu_bolumleme.py — Python

```python
p = a[rastgele_aralik(lo, hi)]
lt, gt = uc_yollu_bolumle(a, lo, hi, p)
# a[lt..gt] = p: eşitler bir daha işlenmez
hizli_uc_yollu(a, lo, lt - 1, d + 1)
hizli_uc_yollu(a, gt + 1, hi, d + 1)
```

Bölümleme iki sınır döndürür: C'de çıkış parametreleri (`&lt, &gt`), C++17'de yapılandırılmış bağlama (`auto [lt, gt]`), Python'da demet. Pivot üç dilde ortak LCG'den.

Üç Dilde: tek tampon · malloc/free · RAII · çöp toplayıcı

H06_02_birlestirmeli_siralama.c — C

```c
int *yrd = malloc(n * sizeof *yrd);
if (yrd == NULL) {
    fprintf(stderr, "bellek yetersiz\n");
    exit(1);
}
tampon_ayirma++;
bs_ozyineli(a, yrd, 0, (int)n - 1, 1);
free(yrd);
```

H06_02_birlestirmeli_siralama.cpp — C++

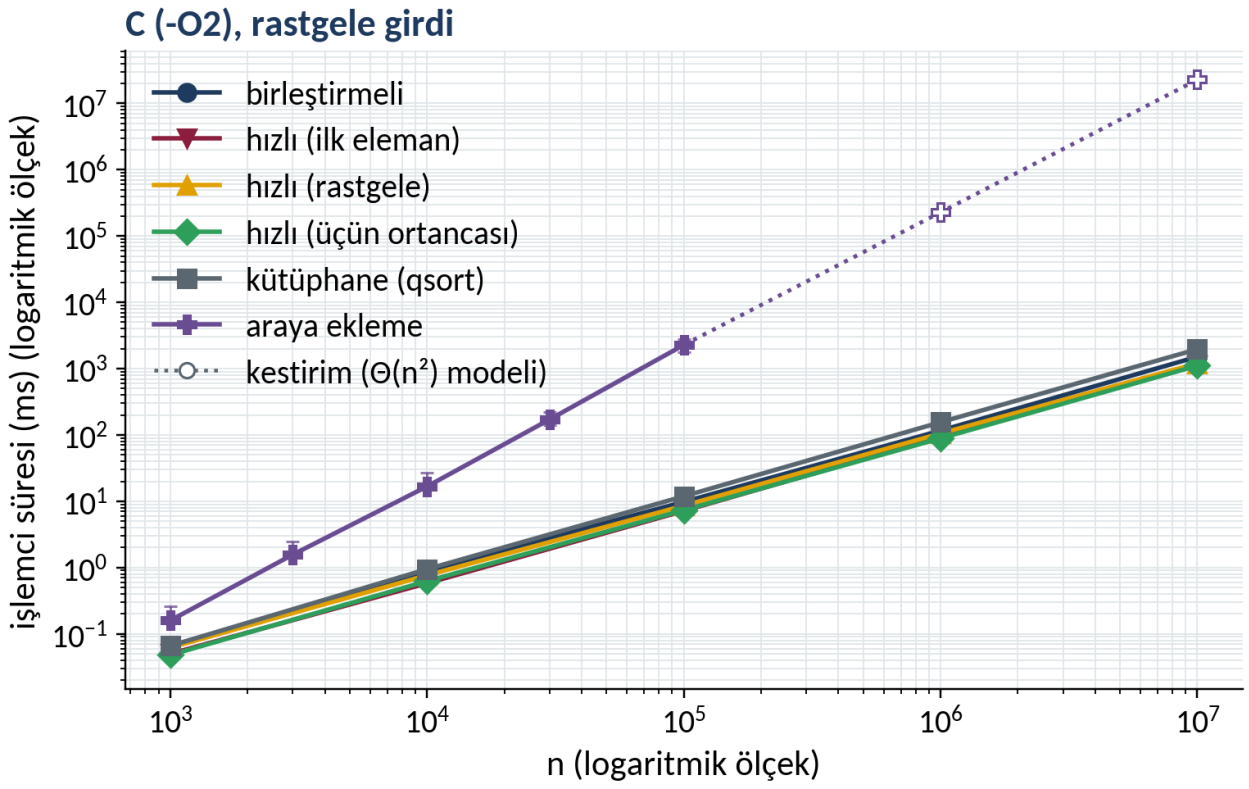
```cpp
std::vector<int> yrd(a.size());
tampon_ayirma++;
int n = static_cast<int>(a.size());
bs_ozyineli(a, yrd, 0, n - 1, 1);
```

H06_02_birlestirmeli_siralama.py — Python

```python
yrd = [0] * len(a)
tampon_ayirma += 1
bs_ozyineli(a, yrd, 0, len(a) - 1, 1)
```

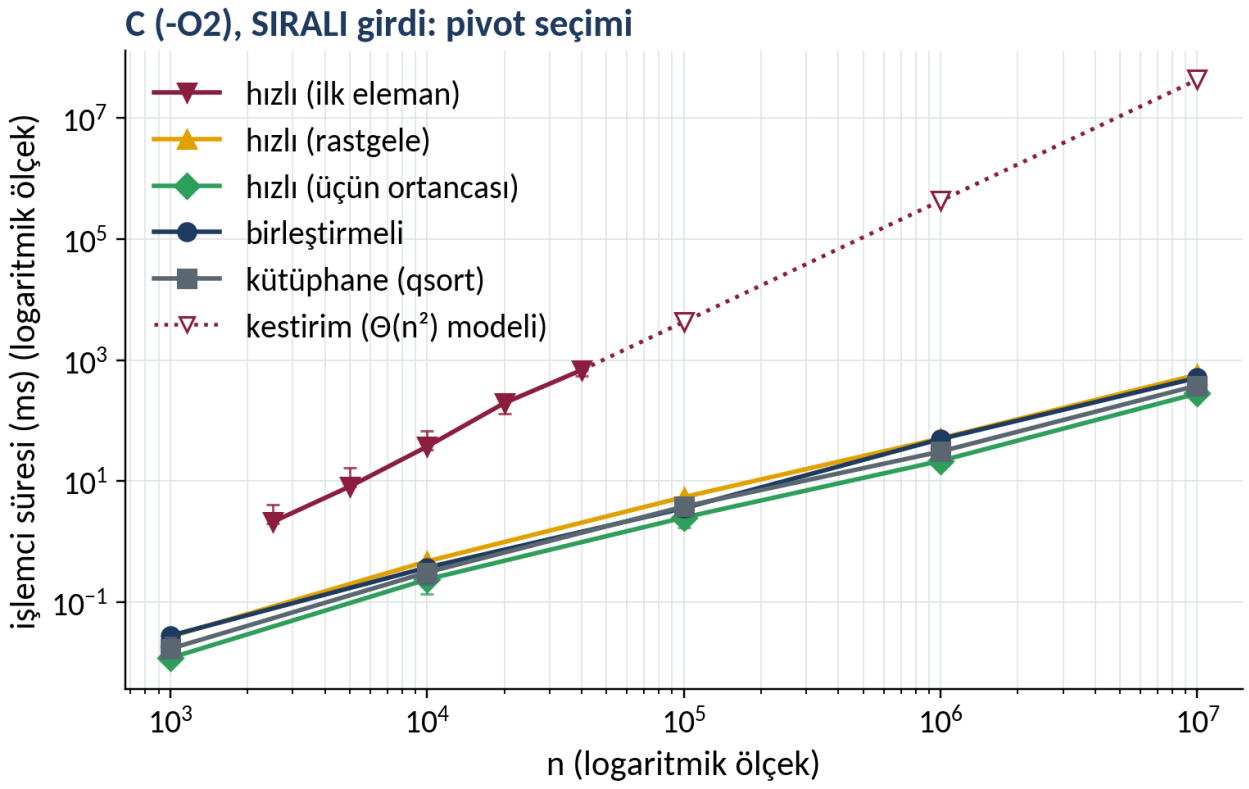
Tampon bir kez ayrılır ve bütün özyinelemeye verilir (her çağrıda ayırıp bırakmamak: HA2). C'de `malloc` + NULL denetimi + `free`; C++'ta `std::vector` kapsam sonunda kendini bırakır (RAII); Python'da çöp toplayıcı.

Haftanın Deneyi: beş n log n eğrisi ve bir n^2



- n = 10^7: hızlı (ortanca3) **1,11 s**, birleştirmeli **1,51 s**, qsort 1,96 s; C++ std::sort 993,5 ms.
- Log–log eğimleri 1,06–1,11 (n log n); araya ekleme ≈ 2.
- Araya ekleme 10^7: ≈ 6,3 sa (**kestirim, alt sınır**).
- Melez eşik etkisi küçük: en iyi eşik **bu makinede ve bu gerçekleştirimde** C'de 24.

Karesel çöküş: sıralı girdi + ilk eleman pivotu

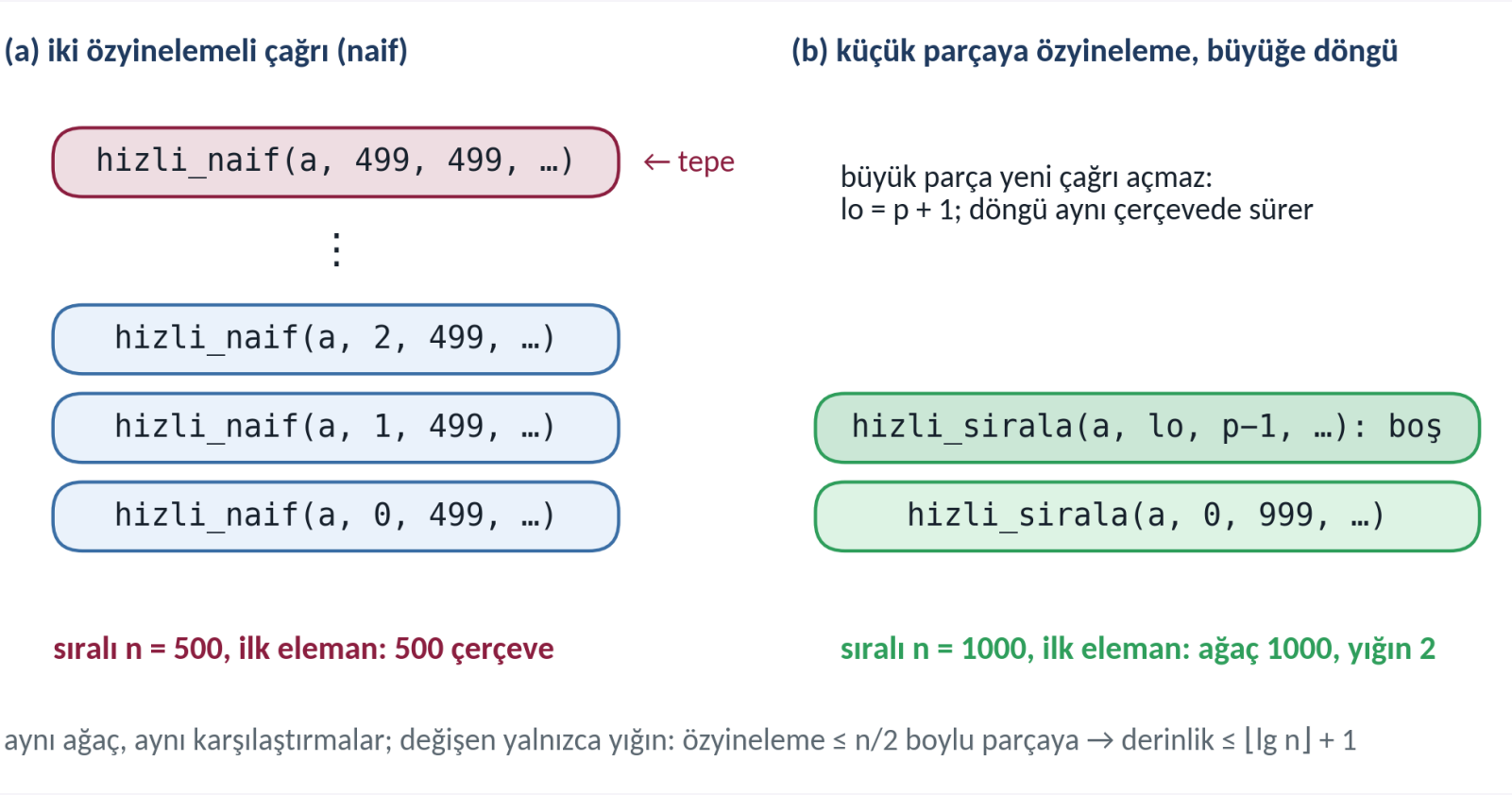


- İlk eleman pivotu: n = 2 500 → 2,11 ms, n = 40 000 → 683,3 ms; eğim ≈ 2.
- Kestirim: n = 10^6 ≈ 7,1 dk, 10^7 ≈ 12 sa (alt sınır).
- Aynı 10^7 sıralı girdi: üçün ortancası 280,6 ms, rastgele 572,3 ms.
- Ağaç yüksekliği 40 000, yığın derinliği **2** (küçük parçaya özyineleme).

KAPUTUN ALTINDA

Yığın derinliği ve kuyruk çağrısı

- İki çağrılı sürümde yığın = **ağaç yüksekliği** (en kötü n).
- 256 KB yığın, ters girdi, n = 20 000: **Segmentation fault** (HA8).
- Sıralı girdi -O2'de çökmez: GCC **kuyruk çağrısını** döngüye çevirir; dilin güvencesi değil.
- Çözüm: **küçük parçaya özyineleme, büyüğe döngü** → yığın ≤ ⌊lg n⌋ + 1.

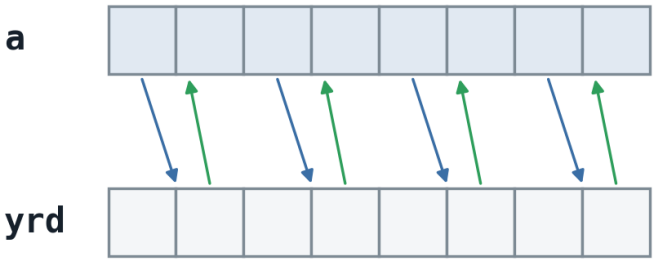


KAPUTUN ALTINDA

Bellek trafiği, önbellek ve kütüphaneler

- Birleştirmeli: her düzeyde tampona gidip dönüş: n = 8'de 48 kopya.
- Hızlı: yerinde, **ardışık** tarama → önbellek dostu.
- C++ `std::sort`: **introsort** (üçün ortancası, derinlik sınırı, 16'nın altında araya ekleme).
- Python `sorted`: **Timsort/Powersort**, kararlı; hızlı sıralama değil.
- C `qsort`: algoritma belirtilmez (bu glibc'de birleştirmeli gibi davrandı).

Birleştirmeli: her düzeyde a → yrd → a



n = 8: 48 kopya = 2·n·lg n

ek O(n) bellek; her eleman her düzeyde iki kez yazılır

Hızlı: dizinin içinde takas



n = 8: 10 takas (Örnek 4)

ek bellek yok (yığın hariç); ardışık tarama önbellek dostu

Kaputun Altında: aynı Θ(n log n), farklı bellek trafiği (Haftanın Deneyi'nde hızlı sıralama daha kısa sürer)

Hata Avcısı: Hoare'un j'si pivotun yeri değil

Hatalı (HA5)

Lomuto alışkanlığı: [lo..p − 1]

H06_HA5_hoare_donus.cpp · satır 37–42 — C++

```
37      if (lo >= hi) {
38          return;
39      }
40      int p = hoare_bolumle(a, lo, hi);
41      hizli_hoare(a, lo, p - 1);          // HATA: a[p] dışarıda kalıyor
42      hizli_hoare(a, p + 1, hi);
```

Düzeltilmiş

Hoare: [lo..j] ve [j + 1..hi]

H06_HA5_hoare_donus_duzeltilmis.cpp · satır 34–39 — C++

```
34      if (lo >= hi) {
35          return;
36      }
37      int p = hoare_bolumle(a, lo, hi);
38      hizli_hoare(a, lo, p);              // p dahil
39      hizli_hoare(a, p + 1, hi);
```

Belirti: `sonuç: 1 3 4 5 2 6 7 8` · 1000 rastgele dizinin (n = 20) 999 tanesi sıralı DEĞİL.

Hata Avcısı: Python'da dilim bir kopyadır

Hatalı (HA3)

a[:p] yeni bir liste

H06_HA3_dilim_kopyasi.py · satır 24–31 — Python

```python
def hizli_sirala(a):
    global kopyalanan
    if len(a) <= 1:
        return
    p = lomuto_bolumle(a)
    kopyalanan += len(a) - 1
    hizli_sirala(a[:p])          # HATA: kopya sıralanır, a değil
    hizli_sirala(a[p + 1:])      # HATA: kopya sıralanır, a değil
```

Düzeltilmiş

aynı liste + (lo, hi)

H06_HA3_dilim_kopyasi_duzeltilmis.py · satır 19–24 — Python

```python
def hizli_sirala(a, lo, hi):
    if lo >= hi:
        return
    p = lomuto_bolumle(a, lo, hi)
    hizli_sirala(a, lo, p - 1)
    hizli_sirala(a, p + 1, hi)
```

Belirti: `sonra: [3, 1, 2, 4, 7, 8, 5, 6]`: yalnızca en üst bölümleme asıl listeye yansır.

Dil Penceresi: C · C++ · Python

Öğe	C	C++	Python
Kütüphane sıralaması	qsort (algoritma belirtilmez)	std::sort (introsort, kararsız) · stable_sort	sorted / list.sort (Timsort, kararlı)
k'ıncı en küçük	elle (quickselect)	std::nth_element	statistics.median (sıralar)
Alt dizi	işaretçi + uzunluk	yineleyici aralığı	a[lo:hi] kopya: O(k)
Tampon	malloc / free	std::vector (RAII)	liste (çöp toplayıcı)
İki değer döndürmek	çıkış parametresi	auto [lt, gt]	demet
Derinlik sınırı	yığın: SIGSEGV	yığın: SIGSEGV	1000: RecursionError
Rastgele pivot	rand()	std::mt19937	random → ortak LCG

Mühendis Gözüyle: en kötü durum bir güvenlik sorunudur

- **McIlroy (1999):** karşılaştırmalara yanıt verirken girdiyi kuran “katil rakip” qsort'ları karesel davranışa zorlar.
- **Crosby ve Wallach (2003):** algoritmik karmaşıklık saldırısı: az veriyle çok işlemci zamanı.
- Savunmalar: **rastgele pivot**, **introsort** (Musser, 1997), garantili birleştirmeli sıralama.
- Kavram 4 (aşağıda): tümü eşit girdi üç stratejiyi birden çökertir → sorun pivotta değil, bölümlemede.
- **Atölye 2 · Pivot avı:** üçün ortancasını, yalnızca farklı tamsayılarla, organ borusunun oranından (0,17) daha yükseğe çıkarın.

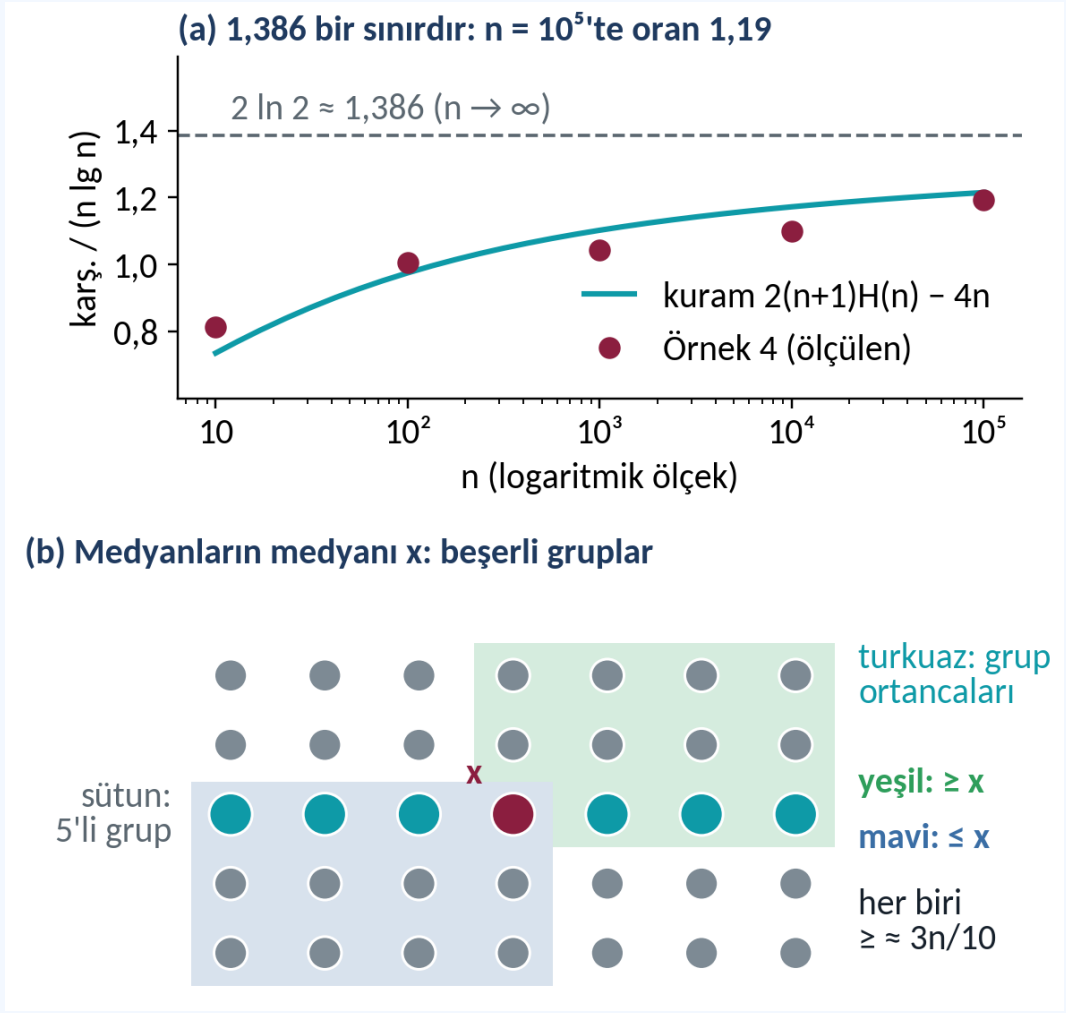
ÇIKTI · KAVRAM 4 (1000 ELEMAN)

```
oran = karşılaştırma / (n(n-1)/2); 1,00 = tam karesel
desen              n        ilk  oran   rastgele  oran   ortanca3  oran
sıralı          1000     499500  1.00      10898  0.02       9009  0.02
ters            1000     499500  1.00      10224  0.02      15773  0.03
organ borusu    1000      84744  0.17      10406  0.02      85752  0.17
tümü eşit       1000     499500  1.00     499500  1.00     502497  1.01
rastgele        1000      10757  0.02      11433  0.02      10759  0.02
```

MERAK KUTUSU

1,39 nereden geliyor? Medyanların medyanı nedir?

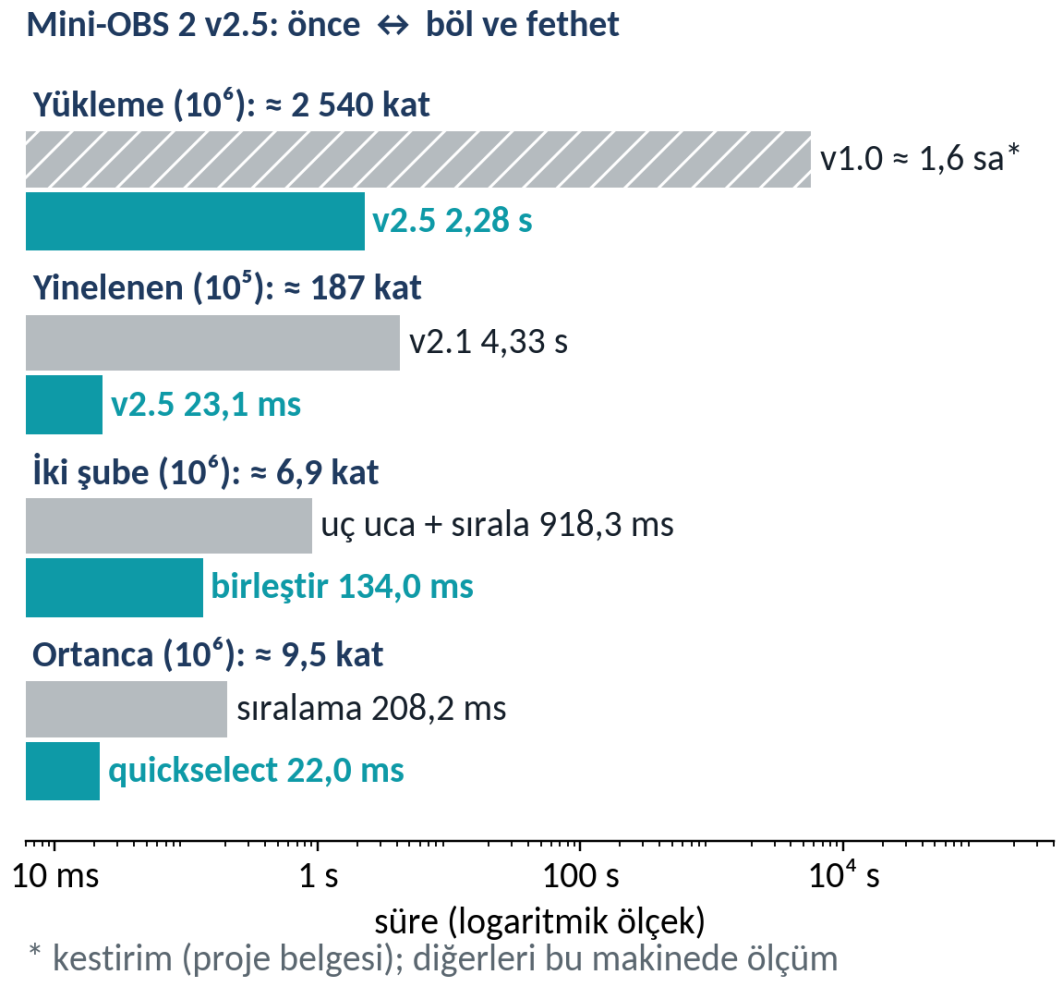
- **Ortalama karşılaştırma:** C(n) = (n − 1) + (2/n)·Σ C(k) → nC(n) = (n + 1)C(n − 1) + 2(n − 1) → **C(n) = 2(n + 1)H(n) − 4n ≈ 2n ln n ≈ 1,386·n lg n**.
- Örnek 4'ün "beklenen" sütunu bu formüldür.
- **Medyanların medyanı:** beşerli grupların ortancalarının ortancası pivot olursa her yanda ≥ ≈ 3n/10 eleman: T(n) ≤ T(n/5) + T(7n/10) + O(n) → **Θ(n)** en kötü durumda da (Blum vd., 1973).
- Sabit çarpanı büyük: pratikte rastgele pivot.



MİNİ-OBS 2 PROJESİ

Mini-OBS 2 v2.5: büyük veride böl ve fethet

- 10^6 kaydı **toplu yükleme**: v1.0'da ≈ 1,6 saat (kestirim) → **2,28 s**.
- Yinelenen numara (10^5): tüm çiftler 4,33 s → sıralayarak **23,1 ms** (187 kat; sonuç aynı).
- İki şube (10^6): birleştirme 134,0 ms ↔ uç uca + sırala 918,3 ms.
- Ortanca (10^6): quickselect 22,0 ms ↔ sıralama 208,2 ms.
- v2.5'in hızlı sıralaması **Hoare** bölümlemesini kullanır; bu haftanın ana sürümü **Lomuto**'dur.



İLERİ BAKIŞ

İleri Bakış: böl ve fethet nerede derinleşiyor?

- **H7:** Ω(n log n) yalnızca karşılaştırmalı sıralamalar için; sayma ve taban sıralaması doğrusal; öbek sıralaması introsort'un yedeği.
- Ana teorem ve yineleme bağıntıları, Algoritma Analizi dersinde Akra–Bazzi gibi genellemelerle sürer.
- Böl ve fethet; hızlı Fourier dönüşümü, Strassen matris çarpımı ve en yakın nokta çifti algoritmalarının temelidir.
- Dış bellek sıralaması: belleğe sığmayan verinin birleştirmeli sıralamayla sıralanması (veritabanı dersleri).
- **H13:** alt problemler örtüşürse dinamik programlama.

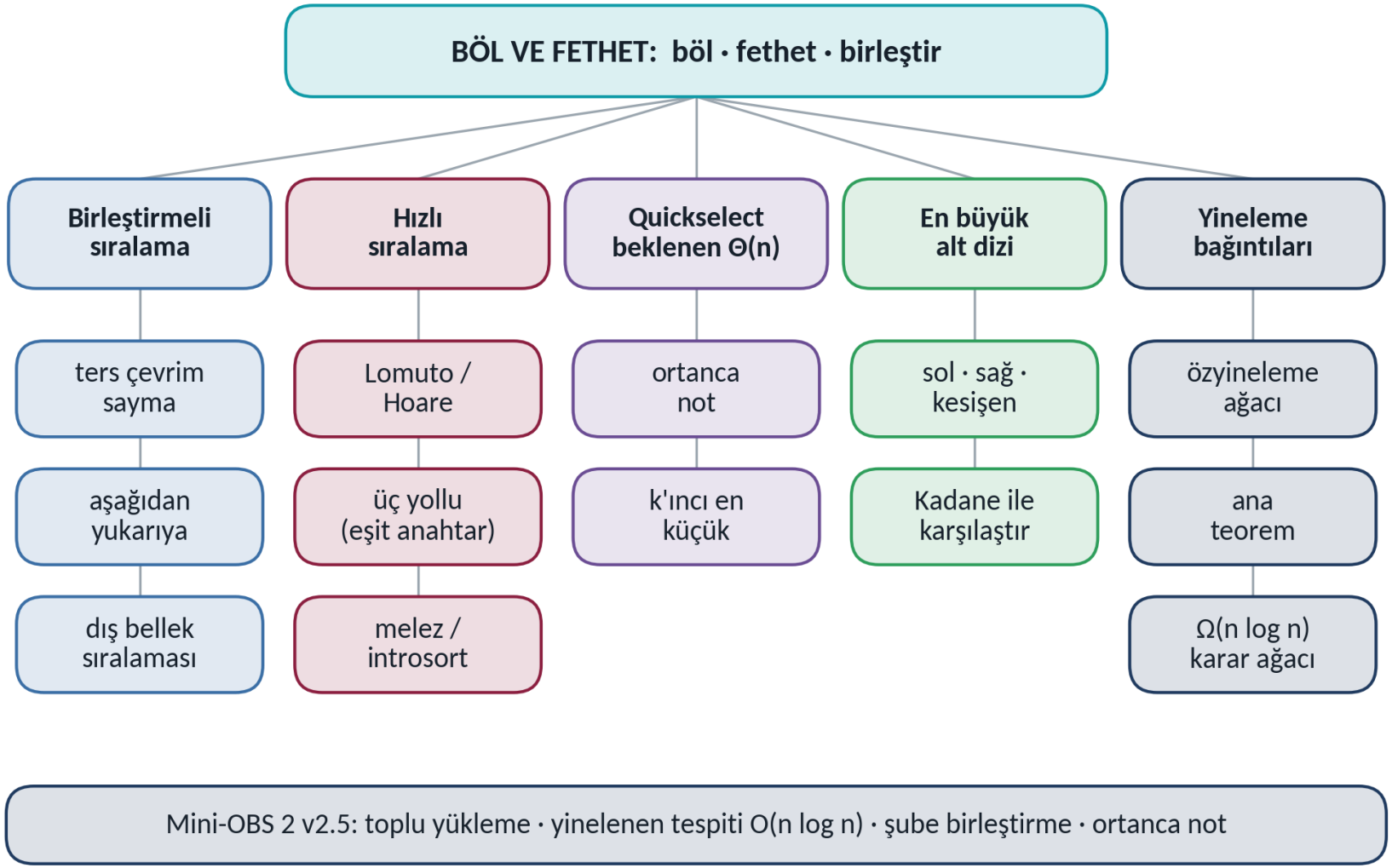
Aynı iskelet: böl · fethet · birleştir

Karatsuba çarpımı `3T(n/2) + n`	**Θ(n^1,585)** Alg. Analizi
Strassen matris çarpımı $7T(n/2) + n^2$	**Θ(n^2,807)** Alg. Analizi
Hızlı Fourier dönüşümü `2T(n/2) + n`	**Θ(n log n)** sinyal işleme
En yakın nokta çifti `2T(n/2) + n`	**Θ(n log n)** geometri
Dengesiz bölme `T(n/3) + T(2n/3) + n`	**Θ(n log n)** Akra–Bazzi
Dış bellek sıralaması `k yollu birleştirme (M: bellek)`	**≈ log_k(n/M) geçiş** veritabanı

Özet: haftanın algoritmaları (kimlik kartı özeti)

Algoritma	En iyi	Ortalama	En kötü	Ek bellek	Not
Birleştirmeli sıralama	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Kararlı · yerinde değil · bağlı listeye uygun
Hızlı sıralama	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ yığın (ortalama)	Kararsız · yerinde · pivot seçimine duyarlı
Quickselect	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$O(1)$ (yinelemeli)	k'ıncı en küçük; ortanca
Ters çevrim sayma (birleştirmeli)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Birleştirme adımında sayım
En büyük alt dizi (böl ve fethet)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$	H2 Kadane ile karşılaştırılır

Kavram haritası



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 **İzleme:** [4, 1, 3, 2] dizisini Lomuto (pivot = a[hi]) ile bölümleyin: p kaçtır, dizi ne olur?

2 **Karmaşıklık:** $T(n) = 2T(n/2) + n^2$ hangi Θ sınıfındadır? Hangi ana teorem durumu?

3 **Hata bulma:** Hoare bölümlemesinin döndürdüğü j ile [lo..j − 1] ve [j + 1..hi] üzerinde özyinelemek neden yanlıştır?

KAPANIŞ

Hazırlık: bir sonraki derse kadar

- **Oku:** ders notu Bölüm 3 (yineleme bağıntıları, ana teorem) ve 4.2 (hızlı sıralama); Ek A'da kendi dilinizin kodları.
- **Çöz:** Alıştırma 6.2 (Lomuto/Hoare izleme), 6.4 (ana teorem), 6.10 (hata bul), 6.13 (Hoare + rastgele pivot).
- **Dene:** web sayfasındaki dört oynatıcı: birleştirme ağacı, bölümleme (Lomuto/Hoare), pivot laboratuvarı, ana teorem hesaplayıcı.
- **Ölç:** `kodlar/deney/` ile Haftanın Deneyi'ni kendi makinenizde tekrarlayın; Atölye 3'ün yarış tablosuna ekleyin.
- **Mini-OBS 2 v2.5:** `bolfethet.c` ve yükleyici; kabul ölçütleri proje belgesinde.
- **H7'ye hazırlık:** öbek (heap) ve sayma sıralaması: karşılaştırmasız sıralama mümkün mü?